



SMART CONTRACT SECURITY ASSESSMENT

Shrooms.fun

SECURITY REVIEW

6 OCTOBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About Shrooms.fun	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	4
5.1	Protocol Summary	5
5.2	Scope	5
6	Findings Summary	5
7	Findings	7

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Shrooms.fun

Shrooms.fun is a community platform for token and NFT holders in which access to rooms is gated by on-chain holdings. Room creators set an entry requirement, the platform checks a user's linked wallets against it, and users who do not meet it can buy the required token directly from the room page. Rooms can be opened for tokens launched through integrated external launchpads such as Pons, or for personal tokens paired with `$SHROOM` and launched through the platform's own launchpad.

The reviewed smart contracts implement the on-chain value flows behind these features. At a high level, the main functionalities are:

- `XTipEscrow` holds native and ERC-20 tips sent to other users or X accounts. A tip is paid to its recipient against a signature from the configured claim signer, subject to per-asset limits and a rolling per-asset claim cap, and is refunded to its sender on expiry
- `CreatorFeeSplitter`, built on the shared `FeeSplitterBase` accounting (also used by `HolderFeeSplitter`), receives a launch's creator-fee revenue from the Pons V2 launcher and fee escrow and splits it 85% to the creator and 15% to the platform treasury, covering both quote-asset fees and vested launch-token buyback proceeds

- The splitter's creator role is transferable, and the creator can schedule a delayed exit that moves the launch's creator-fee recipient to another address after a 7-day notice period
- The in-repository Shrooms V1 launcher (`ShroomsV1LaunchFactory` , the bonding curve, and `ShroomsV1BuybackVault`) models the Pons V2 launch flow, including the five-year buyback vest

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 6 days with a total of 96 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium and three Low severity issues. They're mainly related to the creator exit and role-handoff lifecycle of the fee splitters, the admission rules and documented bounds of the tip escrow's claim cap, the coupling between claim-signer rotation and incident response, and payout accounting for non-standard ERC-20 assets.

The Shrooms.fun team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	SHROOMS.FUN
Repository	shrooms-fun
Type of Project	Token-Gated Community Platform (Fee Splitters & Tip Escrow)
Security Review Timeline	6 days
Review Commit Hash	ee17d94846ff6015e89bd90b4a03b54ea39a3949
Fixes Review Commit Hash	9f97137d24dca8788f4a241f6d53278220d8ddbc

5.2 Scope

The following smart contracts were in the scope of the security review:

FILE	NSLOC
src/CreatorFeeSplitter.sol	144
contracts/src/XTipEscrow.sol	269
src/FeeSplitterBase.sol	185
Total	598

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 1
- **Low** issues: 3
- **Info** issues: 12

ID	TITLE	SEVERITY	STATUS
[M-01]	Creator Exit Redirects the Treasury's Share of the Remaining Vest and Unswept Fees	MEDIUM	ACKNOWLEDGED
[L-01]	<code>claimCap == maximum</code> Lets Ordinary Claim Traffic Starve Maximum-Sized Tips Until They Refund	LOW	FIXED
[L-02]	Full-Balance Splitter Payouts Can Strand Revenue in a Per-Transfer-Capped Quote Token	LOW	FIXED

ID	TITLE	SEVERITY	STATUS
[L-03]	<code>bindLaunch()</code> Can Become Impossible, Stranding Launch-Token Revenue and Blocking Exit	LOW	FIXED
[I-01]	Splitter Events and Views Do Not Expose the Full Revenue State	INFO	FIXED
[I-02]	Signer Rotation and Claim Resumption Coupled, Extending Outages and Ending Freezes Automatically	INFO	FIXED
[I-03]	Exit Lifecycle Checks Are Inconsistent Before and After the First Exit	INFO	FIXED
[I-04]	Partial Claim Wrappers Accept Zero Amounts and Ignore the Escrow's Reported Payment	INFO	FIXED
[I-05]	<code>XTipEscrow</code> Cap Views Report Incompatible Versions of Current Claim Capacity	INFO	FIXED
[I-06]	Reusing a Splitter Can Strand Revenue or Assign a Launch to the Wrong Creator	INFO	ACKNOWLEDGED
[I-07]	Splitter Deployment Does Not Verify That the Fee Escrow Belongs to the Factory	INFO	FIXED
[I-08]	<code>FeeSplitterBase</code> Clears ERC-20 Liabilities Without Verifying Exact Payout	INFO	FIXED
[I-09]	The Owner Can Bypass the Pauser Veto and Redirect Every Live Tip to Itself	INFO	FIXED
[I-10]	The Claim Cap Lets About Twice <code>claimCap</code> Out Within One Window	INFO	FIXED
[I-11]	Outgoing Creator Can Execute a Ready Exit Ahead of <code>acceptCreator()</code>	INFO	FIXED
[I-12]	A Scheduled Exit Never Expires	INFO	FIXED

7 Findings

M-01

MEDIUM

ACKNOWLEDGED

Creator Exit Redirects the Treasury's Share of the Remaining Vest and Unswept Fees

Description

`CreatorFeeSplitter` allocates covered revenue 85% to the creator and 15% to the platform treasury. Despite that shared ownership, `executeExitIfCurrent()` lets the creator transfer the launch's complete creator-fee recipient identity to an arbitrary address without first settling value that already belongs partly to the treasury.

The launcher resolves payment destinations when it pays. Transferring the recipient changes the address used by the bonding curve, graduated pool hook, and buyback vault. The vault also changes the beneficiary of the unreleased five-year vest. Fees earned before the exit but not yet swept follow the new recipient as well.

The splitter's local ledger remains solvent because these assets never reach it. The launcher redirects them before `_accrue()` can recognise them.

The seven-day exit delay protects only a small fraction of a fresh five-year vest. About `7/1825`, or 0.384%, vests during that period. The creator can execute the exit before a competing release in the same block. Shrooms has no recovery function after the handoff. The in-repository launcher disables owner recovery entirely. The live Pons V2 factory keeps an owner override with a three-day timelock (`setCreatorFeeRecipient()` and `executeCreatorFeeRecipientChange()`); `CREATOR_FEE_RECIPIENT_TIMELOCK` reads 259200 on chain, so reversing an exit there depends on Pons' owner, a third party.

Each piece of this behavior is documented and intended on its own. The owner's 2026-09-21 decisions keep the full seven-day exit with no lock-in and note that the buyback passthrough turns part of the treasury's quote revenue into vested launch tokens (`docs/CONTRACT_REVIEW_FIXES_2026-09-21.md:10,44`). The authority decisions state that a creator-authorized transfer also moves accrued, unreleased vesting (`docs/CONTRACT_AUTHORITY_DECISIONS_2026-09-18.md:29`). No document covers the combination: the treasury's share of vest principal locked before the exit leaves with the creator. The second audit says the seven-day exit "does not erase accrued amounts" (`docs/audits/2026-09-18-SHROOMS_V1_SECOND_AUDIT.md:133`). That is true for escrow credits, but not for unreleased vest principal or fees still pending at exit.

Location of Affected Code

File: [contracts/src/CreatorFeeSplitter.sol#L154-L162](#)

```
function executeExitIfCurrent(address expectedRecipient, uint256 expectedExitAt) external nonReentrant {
    if (msg.sender != creator) revert Unauthorized();
    _requireCurrentExit(expectedRecipient, expectedExitAt);
    if (block.timestamp < expectedExitAt) revert ExitNotReady();
    exitAt = 0;
    exitRecipient = address(0);
    ponsFactory.transferCreatorFeeRecipient(token, expectedRecipient);
    emit Exited(expectedRecipient);
}
```

File: [contracts/launcher/src/ShroomsV1LaunchFactory.sol#L906-L924](#)

```
function _setCreatorFeeRecipient(address token, LaunchedToken storage launch, address newRecipient) private {
    // code
    launch.creatorFeeRecipient = newRecipient;

    if (launch.phase == GraduationPhase.PoolCreated) {
        memeHook.setCreatorFeeRecipient(_poolIdFor(token, launch), newRecipient);
    } else {
        ShroomsV1BondingCurve[launch.curve].setCreatorFeeRecipient(newRecipient);
    }

    buybackVault.updateCreatorRecipient(token, newRecipient);
}
```

File: [contracts/launcher/src/ShroomsV1BuybackVault.sol#L176-L196](#)

```
function release(address token) external nonReentrant returns (uint256 released) {
    if (factory == address(0)) revert NotFactory();

    LaunchVest storage v = _vaults[token];
    if (msg.sender != v.creatorRecipient && msg.sender != v.protocolRecipient) revert NotVestBeneficiary();

    _checkpoint(v, block.timestamp);
    released = v.vestedUnreleased;
    if (released == 0) return 0;
    v.vestedUnreleased = 0;
    v.totalReleased += released;

    uint256 protocolAmount = (released * v.protocolFeeShareBps) / BASIS_POINTS;
    uint256 creatorAmount = released - protocolAmount;

    IERC20(token).forceApprove(address(feeEscrow), released);
}
```

```

if (protocolAmount != 0) feeEscrow.creditToken(v.protocolRecipient, token, protocolAmount);
if (creatorAmount != 0) feeEscrow.creditToken(v.creatorRecipient, token, creatorAmount);

emit Released(token, creatorAmount, protocolAmount);
}

```

Impact

The treasury loses $0.105 \times V$ of the unreleased buyback principal, where V is the affected vest principal. It also loses $0.15 \times U$ of creator-side fees earned but unswept at exit and its 15% allocation from all covered post-exit revenue.

With the default 1% launch fee, 35% buyback earmark, 70% creator vest leg, and 15% treasury split, the vest component is approximately 0.03675% of cumulative launch volume.

The executable test measured a treasury vest loss of 217,653,908,930,888,515,206 launch-token wei. For the fees left unswept at exit, the test's `treasuryQuoteLoss` value of 0.105 ether is 15% of their 0.7 ether creator-side value. Half of that value is paid in quote, so the treasury's quote loss is 0.0525 ether. The other half funds a new buyback lock, and the treasury loses 15% of the creator leg of those launch tokens. The creator needs only the documented schedule-and-execute flow. Against the in-repository launcher, the loss is permanent: the owner-recovery selectors revert and the splitter is no longer the current recipient. On live Pons V2, the only way back is the Pons owner's three-day override, which Shrooms does not control.

Proof of Concept

The executable test is `contracts/test/poc/F001VestRedirect.t.sol`. It deploys the in-repository launcher, curve, escrow, vault, and splitter.

```

// Fund a real five-year buyback vest for the splitter.
vm.startPrank(BUYER);
quote.approve(c, 200 ether);
curve.buy(100 ether, 1, BUYER);
vm.stopPrank();
curve.sweepFees(1);

uint256 locked = vault.totalLocked(t);
require(locked > 0, "vest funded by the curve buyback");

// Leave fee-bearing activity unswept across the exit.
vm.startPrank(BUYER);
curve.buy(100 ether, 1, BUYER);
vm.stopPrank();

vm.prank(CREATOR);
splitter.scheduleExit(EXITR);

```

```

uint256 exitAt = block.timestamp + splitter.EXIT_DELAY();
vm.warp(exitAt);

vm.prank(CREATOR);
splitter.executeExitIfCurrent(EXITR, exitAt);

// The splitter can no longer release the vest.
vm.prank(SHROOMS_TREASURY);
pocVm.expectRevert(ShroomsV1BuybackVault.NotVestBeneficiary.selector);
splitter.releaseVestedLaunchTokens();

// The exit address receives the remaining creator leg directly.
vm.warp(exitAt + vault.VESTING_DURATION() - 7 days + 1);
vm.prank(EXITR);
vault.release[t];

uint256 r2 = locked - r1;
uint256 creatorLeg2 = r2 - (r2 * 3000) / 10000;
uint256 treasuryVestLoss = (creatorLeg2 * 1500) / 10000;

require(escrow.balanceOfToken(EXITR, t) == creatorLeg2, "EXITR takes the full remaining creator leg");
require(escrow.balanceOfToken(address(splitter), t) == 0, "splitter receives zero of the remaining vest");
require(treasuryVestLoss > 0, "treasury loss equals 15% of the redirected creator leg");

// Fees earned before exit follow the new recipient when swept afterward.
uint256 splitterQuoteLedgerBefore = splitter.lifetimeReceived();
curve.sweepFees[1];
require(escrow.balanceOfToken(EXITR, address(quote)) == 0.35 ether, "unswept creator quote leg pays EXITR");
require(
    splitter.lifetimeReceived() == splitterQuoteLedgerBefore,
    "splitter quote ledger unchanged by the post-exit sweep"
);

```

Run:

```

cd contracts
forge test --match-test test_F_001 -vv

```

Observed:

```

[PASS] test_F_001_ExitRedirectsRemainingVestAndUnsweptFees()
Gas: 5,769,870

vestLocked:      2080875812241979885021
redirectedCreatorLeg: 1451026059539256768043
treasuryVestLoss: 217653908930888515206

```

treasuryQuoteLoss: 105000000000000000

treasuryRescuedSliver: 838051354519372720

A fork test against the deployed Pons V2 contracts on Robinhood Chain

(`contracts/test/validation/Issue207.t.sol`) shows the same redirect in production: after the exit, the vest's creator beneficiary is the exit recipient, pre-exit fees swept afterward are credited to it, and the splitter receives neither.

The in-repository test also confirms that the treasury cannot restore the old recipient. Its transfer attempt reverts with `NotCreatorFeeRecipient`, and all three legacy recovery selectors revert with `CreatorRecoveryDisabled`.

Recommendation

Run a best-effort internal settlement pass before exit, then refuse the handoff while any unreleased buyback principal, pending curve fee, pending creator tax, or pending hook bucket remains. Isolate every external settlement call with `try/catch`; an empty or temporarily reverting dependency must not trap an otherwise clean exit. Fail closed if the required external state cannot be read.

Do not use the splitter's local `platformOwed` or `creatorOwed` balances as the refusal condition. Those balances survive the handoff and are not the assets at risk. Do not push accrued balances during exit, since an unreceivable beneficiary would then block exit permanently. Implement settlement with internal helpers; an external self-call to another `nonReentrant` function will fail under the active guard.

This change may delay exit until a five-year vest completes or until the relevant Pons operator settles new pending fees. Preserve the restriction against an administrative recipient override or custody escape hatch.

Team Response

Acknowledged.

L-01

● LOW

FIXED

`claimCap == maximum` **Lets Ordinary Claim Traffic**
Starve Maximum-Sized Tips Until They Refund

Description

`XTipEscrow` limits claims with a per-asset leaky bucket. A claim that arrives while the bucket is empty is always admitted. Once `used` is nonzero, the whole claim must fit inside `claimCap - used`.

`setAsset()` accepts `claimCap == maximum`, and its NatSpec states only that the cap "must cover one maximum tip". At that boundary, a maximum-sized tip has zero headroom: it can be admitted only in a moment when the bucket is completely empty. Smaller claims do not have this constraint, because they fit into partial headroom.

No attacker is needed. When ordinary claim traffic for the asset runs near the configured throughput (`claimCap` per `CLAIM_CAP_WINDOW`), the bucket is almost never empty. Every smaller claim keeps succeeding and `claimCapAvailable()` keeps reporting most of the cap as available, yet each attempt to claim a maximum-sized tip reverts with `ClaimCapReached`. Because the drain never catches up with the incoming traffic, this lasts as long as the traffic does. If it covers the tip's lifetime, the tip expires and refunds to its sender instead of paying the recipient.

The 2026-09-21 review treats the claim cap as a shared rate limit whose delay is bounded because the bucket drains "fully within an hour" (`docs/CONTRACT_REVIEW_FIXES_2026-09-21.md:43`). It also says that "a claim always fits an empty bucket, so lowering limits later cannot strand an older, larger tip" (`docs/CONTRACT_REVIEW_FIXES_2026-09-21.md:38`). Neither bound holds at `claimCap == maximum` under sustained traffic. The cap is never exhausted, so the one-hour drain never completes, and the empty-bucket guarantee applies only at the instants when the bucket touches zero. The same review recommends setting caps "with headroom", but the contract permits zero headroom and its documentation implies that equality is enough.

This issue does not rely on dust grieving. A single small claim drains within about a second at realistic caps, so blocking a maximum claim deliberately would require winning ordering against every retry. The issue is the admission rule under ordinary traffic in a configuration the contract explicitly allows.

Location of Affected Code

File: [contracts/src/XTipEscrow.sol#L159-L169](#)

```
/// @param claimCap Rolling cap per 'CLAIM_CAP_WINDOW'; must cover one maximum  
/// tip. Use 'type(uint256).max' for no cap.  
function setAsset(address asset, uint256 minimum, uint256 maximum, uint256 claimCap, bool supported)  
    external  
    onlyOwner  
{  
    if (minimum == 0 || maximum < minimum || claimCap < maximum) revert InvalidInput();  
    ...  
}
```

File: [contracts/src/XTipEscrow.sol#L386-L394](#)

```
function _consumeClaimCap(address asset, uint256 amount) private {  
    uint256 cap = assets[asset].claimCap;  
    uint256 used = _bucketUsed(asset, cap);
```

```

if (used != 0 && (amount > cap || used > cap - amount)) revert ClaimCapReached(used >= cap ? 0 : cap - used);
claimBuckets[asset] = ClaimBucket(used + amount, uint64(block.timestamp));
}

```

With `cap == maximum` and `amount == maximum`, `cap - amount == 0`, so any `used > 0` rejects the claim.

Impact

For assets configured at `claimCap == maximum`, maximum-sized tips are starved whenever claim traffic runs near the configured rate. Smaller tips settle normally over the same period. In the worst case, the starvation covers the tip's remaining lifetime, and the recipient loses the payment to an expiry refund. Near-maximum tips are affected in proportion to how little headroom they leave.

The tip remains fully backed and the escrow's custody is unaffected. The condition arises only for assets configured at or near `claimCap == maximum` and depends on claim volume. It persists while traffic continues, with no attacker involvement. Because `setAsset()` accepts this configuration and its NatSpec presents it as sufficient, an operator following the contract's documented constraint can deploy an asset in which the largest tips cannot be claimed reliably.

Proof of Concept

Configuration: native asset, `maximum = claimCap = 10 ETH`. Honest 1 ETH tips are claimed every 6 minutes, exactly the configured throughput of 10 ETH per hour. After each small claim, the maximum tip's recipient retries 1 second, 3 minutes, and 5 minutes 59 seconds later. The test repeats this for 24 hours. The comparison test runs the same traffic with `claimCap = 2 × maximum`.

```

uint256 constant MAX = 10 ether;
uint256 constant SMALL = 1 ether;
uint256 constant GAP = 6 minutes; // SMALL * CLAIM_CAP_WINDOW / cap
uint256 constant CYCLES = 240; // 24 hours

function _tryMax(bytes32 id, uint256 t) internal {
    vm.warp(t);
    uint256 dl = t + 300;
    bytes memory s = sigWith(KEY, id, RECIP, dl);
    try escrow.claim(id, RECIP, dl, s) {
        revert("max claim admitted");
    } catch (bytes memory err) {
        require(bytes4(err) == XTipEscrow.ClaimCapReached.selector, "wrong revert");
    }
}

function _run(uint256 cap) internal returns (bytes32 big, uint256 blocked) {
    escrow.setAsset(address(0), 1, MAX, cap, true);
    big = tipNative(SENDER, MAX, uint64(TO + 1 days), 0);
    for (uint256 i; i < CYCLES; i++) {

```

```

uint256 t = TO + i * GAP;
vm.warp(t);
bytes32 small = tipNative(SENDER, SMALL, uint64(t + 1 days), i + 1);
_claim(small, address(uint160(0x1000 + i)), t); // honest traffic, always admitted
if (cap == MAX) {
    _tryMax(big, t + 1);
    require(escrow.claimCapAvailable(address(0)) >= MAX - SMALL, "view shows ample room");
    _tryMax(big, t + GAP / 2);
    _tryMax(big, t + GAP - 1);
    blocked += 3;
} else if (i == 0) {
    vm.warp(t + 1);
    _claim(big, RECI, t + 1); // same state, admitted immediately
}
}
}

function test_maxTipStarvesUnderFullRateHonestTraffic() public {
    (bytes32 big, uint256 blocked) = _run(MAX);
    require(blocked == 720, "every attempt over 24h rejected");
    vm.warp(TO + 1 days);
    escrow.refund(big);
    require(RECI.balance == 0, "recipient never paid");
}

function test_headroomAdmitsSameTraffic() public {
    _run(2 * MAX);
    require(RECI.balance == MAX, "admitted with 2x headroom");
}

```

`tipNative` creates a native tip and `sigWith` signs the claim digest with the attester key.

Observed:

```

[PASS] test_headroomAdmitsSameTraffic()
[PASS] test_maxTipStarvesUnderFullRateHonestTraffic()

```

All 720 retries over 24 hours reverted with `ClaimCapReached`, while `claimCapAvailable()` reported at least 9 ETH available after every small claim. All 240 small claims settled. The maximum tip then expired and refunded to its sender. With `claimCap = 2 × maximum`, the same tip was admitted one second after the first small claim.

The bucket reaches zero only in the same second as the next small claim, so a retry has to land in that exact second, ordered ahead of the small claim. At lower traffic, the empty windows grow in proportion. At traffic near the cap, they effectively disappear.

Recommendation

Enforce headroom in `setAsset()` rather than relying on operator guidance, while keeping `type(uint256).max` as the uncapped sentinel:

```
if (
  minimum == 0 || maximum < minimum
  || (claimCap != type(uint256).max && claimCap / 2 < maximum)
) revert InvalidInput();
```

With this change, a maximum-sized tip is admitted whenever the bucket is at most half full, instead of only when it is empty. Ordinary traffic within the configured rate then no longer starves it, as the comparison test shows. Correct the `claimCap` NatSpec to state the requirement, and note that the cap also sets the loss bound if the attester is compromised, so operators size `maximum` down rather than the cap up.

Also correct the comment above `_consumeClaimCap()`. Admission of a maximum or oversized tip is guaranteed only when the bucket is empty, not under ongoing traffic.

Do not add partial payouts: tips are designed to settle once for their full value.

Team Response

Fixed.

L-02

● LOW

FIXED

Full-Balance Splitter Payouts Can Strand Revenue in a Per-Transfer-Capped Quote Token

Description

`FeeSplitterBase` deliberately supports quote and launch tokens that cap the amount of a single transfer. Its partial escrow-claim wrappers exist for that reason: the NatSpec says a third party could grow the escrow balance past a token's per-transfer limit, and that partial claims keep the revenue reachable.

The payout side does not follow the same rule. Every accrued piece is added to one cumulative owed balance per role, and `_payBeneficiary()` and `_payPlatform()` send that entire balance in a single transfer. `withdraw()` and `withdrawLaunchTokens()` take no amount, and the splitter has no partial payout, rescue, or upgrade path.

Once a role's owed balance exceeds the token's per-transfer maximum, its withdrawal reverts. The revert restores the owed balance, so every retry attempts the same oversized transfer. The balance can cross the cap through ordinary fee accrual between withdrawals. A third party can also force it by transferring up to the cap directly to the splitter, as many times as needed, and then calling the permissionless `accrue()`.

This is the payout-side remainder of the transfer-limit constraint identified in the 2026-09-18 second audit. Partial escrow claims fixed ingress. Egress is still full-balance only.

Location of Affected Code

File: [contracts/src/FeeSplitterBase.sol#L199-L206](#)

```
/// @notice As 'claimAndAccrue', for a chosen amount. Escrow credit is  
/// permissionless, so a third party could grow the balance past a token's  
/// per-transfer limit; partial claims keep the revenue reachable.  
function claimAmountAndAccrue(uint256 amount) external nonReentrant {  
    if (asset == address(0)) feeEscrow.claim(amount);  
    else feeEscrow.claimToken(asset, amount);  
    _accrue(asset);  
}
```

File: [contracts/src/FeeSplitterBase.sol#L331-L359](#)

```
function _payPlatform(address trackedAsset) internal {  
    _accrue(trackedAsset);  
    Ledger storage l = _ledgers[trackedAsset];  
    uint256 amount = l.platformOwed;  
    l.platformOwed = 0;  
    _send(trackedAsset, platform, amount);  
}  
  
function _payBeneficiary(address trackedAsset, address to) internal {  
    _accrue(trackedAsset);  
    Ledger storage l = _ledgers[trackedAsset];  
    uint256 amount = l.beneficiaryOwed;  
    l.beneficiaryOwed = 0;  
    _send(trackedAsset, to, amount);  
}
```

File: [contracts/src/CreatorFeeSplitter.sol#L76-L84](#)

```
function withdraw() external nonReentrant {  
    _withdraw(asset);  
}
```

```
function withdrawLaunchTokens() external nonReentrant {
    _withdraw(_boundToken());
}
```

`HolderFeeSplitter` inherits the same full-balance helpers.

Impact

For a splitter that serves a per-transfer-capped asset, the creator's or treasury's accrued share can become permanently unwithdrawable. A griever can lock both roles by donating slightly more than the cap in total, in pieces no larger than the cap. The donated value is also trapped. The splitter is immutable, and neither role transfer nor exit can move an existing owed balance.

Applicability is conditional. Live Pons pair approval checks token code, economics, and decimals, but not a transfer maximum, so such a token can be approved. The currently identified quote token (SHROOM, `0xab093dEF657F15dF31b33922A95e047aDd645B29`) exposes no transfer-cap getter, and no cap is documented. A small cap would also obstruct Pons' own large transfers, so a practical cap would be large.

Proof of Concept

The executable test is `contracts/test/validation/PozCapPayout.t.sol`. It uses an ERC-20 whose only non-standard rule is `amount <= 100` for non-mint, non-burn transfers.

```
function testPartialEscrowClaimCannotBePaidOutWhenOwedExceedsTransferCap() public {
    PozCappedQuote quote = new PozCappedQuote();
    PozCapPons pons = new PozCapPons(quote);
    CreatorFeeSplitter splitter =
        new CreatorFeeSplitter(CREATOR, TREASURY, address(quote), address(pons), address(pons));

    pons.addCredit(address(splitter), 300);
    splitter.claimAmountAndAccrue(100);
    splitter.claimAmountAndAccrue(100);
    splitter.claimAmountAndAccrue(100);
    require(splitter.creatorOwed() == 255 && splitter.platformOwed() == 45, "cumulative split");

    vm.prank(CREATOR);
    vm.expectRevert();
    splitter.withdraw();
    require(splitter.creatorOwed() == 255, "creator claim remains trapped");
}

function testSmallDonationsCanTrapBothPayouts() public {
    // seven permitted 100-unit donations, then accrue()
    require(splitter.creatorOwed() == 595 && splitter.platformOwed() == 105, "both shares exceed cap");
}
```

```
// both creator and treasury withdraw() revert; 700 units remain inaccessible  
}
```

Run:

```
cd contracts  
forge test --match=contract PozCapPayoutTest -vv
```

Observed:

```
[PASS] testPartialEscrowClaimCannotBePaidOutWhenOwedExceedsTransferCap() (gas: 1301772)  
[PASS] testSmallDonationsCanTrapBothPayouts() (gas: 1349857)
```

Recommendation

Add bounded partial payouts, for example, `withdraw(uint256 amount)` and `withdrawLaunchTokens(uint256 amount)`, plus a partial holder forward. Each should debit no more than the caller's owed balance. Recipients must stay role-bound, and the cumulative 15% allocation must not change. Test partial payouts, failed transfers, and the owed-balance invariant.

Alternatively, declare per-transfer-capped assets unsupported, check this at activation, and remove the per-transfer-limit statement from the `claimAmountAndAccrue()` NatSpec. Already deployed splitters cannot gain a partial-payout function.

Team Response

Fixed.

L-03

LOW

FIXED

`bindLaunch()` Can Become Impossible, Stranding
Launch-Token Revenue and Blocking Exit

Description

Binding is required for:

- launch-token revenue (`accrueLaunchTokens()`, `claimLaunchTokens*`, `withdrawLaunchTokens()`);
- exit;
- every Pons passthrough.

It can happen only once, only by the creator, and only if Pons' current record shows `creatorFeeRecipient == address(this)` and `pairToken == asset`. There are two realistic ways it becomes permanently impossible:

(a) Quote mismatch. The splitter is created with one quote asset (say WETH) and named as the recipient of a launch paired with another (native ETH).

- `bindLaunch()` always reverts with `LaunchMismatch`.
- The native revenue credited to the splitter in Pons' escrow can never be claimed, because the splitter only ever asks the escrow for WETH.
- The creator can never move the recipient, because exit requires a bound launch. Only Pons' owner can.

(b) Late binding. The NatSpec says binding is "Needed only for launch-token revenue, exit and the Pons passthroughs; quote-asset revenue can be claimed and split without it", so a creator may reasonably not bind. Then:

1. Launch-token revenue is credited to the splitter. For example, Pons' protocol beneficiary calls `vault.release()`, which credits the splitter's share in the escrow.
2. Pons' owner redirects the recipient.
3. `bindLaunch()` now reverts forever, and nobody can ever claim those tokens.

The creator can avoid (b) by binding during Pons' 3-day notice period

(`test_V_LO3b_creatorCanAvertByBindingDuringTimelock`). The platform cannot bind to rescue its own share (`test_V_LO3_platformCannotBind`).

Location of Affected Code

File: [contracts/src/FeeSplitterBase.sol#L169-L179](#)

```
function bindLaunch(address launchToken) external {
    if (msg.sender != _binder()) revert Unauthorized();
    if (token != address(0)) revert AlreadyBound();
    IPonsFactory.Launch memory launch = ponsFactory.getLaunchedToken(launchToken);
    if (
        launchToken == address(0) || launchToken == asset || !launch.exists || launch.token != launchToken
        || launch.creatorFeeRecipient != address(this) || launch.pairToken != asset
    ) revert LaunchMismatch();
    token = launchToken;
    emit Bound(launchToken, msg.sender);
}
```

File: [contracts/src/CreatorFeeSplitter.sol](#)

Impact

- **(a):** all of that launch's revenue is stuck in the escrow, and the creator cannot redirect it.
- **(b):** launch-token revenue already credited to the splitter is stranded permanently.

Both require a misconfiguration or an inactive creator.

Proof of Concept

```
/// (a) Splitter created with a quote that differs from the launch's pair.
function test_L03a_quoteMismatch_revenueAndRecipientLocked() public {
    MockERC20 weth = new MockERC20("Wrapped Ether", "WETH");
    CreatorFeeSplitter s = _splitter(address(weth));
    (MockERC20 lt, CurveDouble curve) = _launch(address(s), address(0), false); // native pair

    curve.takeTradeFee{value: 10 ether}();
    vm.prank(operator);
    curve.sweepFees(0);
    assertEq(escrow.balanceOf(address(s)), 7 ether);

    vm.prank(creator);
    vm.expectRevert(LaunchMismatch.selector);
    s.bindLaunch(address(lt));
    vm.prank(creator);
    vm.expectRevert(NotBound.selector);
    s.scheduleExit(creatorWallet);
    vm.expectRevert(PonsV2FeeEscrow.NoBalance.selector); // only ever asks the escrow for WETH
    s.claimAndAccrue();
    vm.prank(creator);
    vm.expectRevert(PonsFactoryDouble.NotCreatorFeeRecipient.selector);
    pons.transferCreatorFeeRecipient(address(lt), creatorWallet);
    // 7 ETH (and all future native revenue of this launch) can never leave the escrow
    // for anyone, only Pons' owner can redirect future revenue.
}

/// (b) Launch-token revenue reaches the splitter before it is bound (binding is
/// documented as optional for quote revenue), then Pons' owner redirects the
/// recipient: that revenue is stranded for good.
function test_L03b_ownerRedirectBeforeBind_strandsLaunchTokenRevenue() public {
    CreatorFeeSplitter s = _splitter(address(0));
    (MockERC20 lt, CurveDouble curve) = _launch(address(s), address(0), true); // buyback on at launch

    curve.takeTradeFee{value: 10 ether}();
    vm.prank(operator);
    curve.sweepFees(1); // locks 3.5 ETH worth of LT for the splitter

    vm.warp(block.timestamp + 365 days);
    vm.prank(ponsProtocol); // Pons' own beneficiary may release; creator share is credited to the splitter
    vault.release(address(lt));
    uint256 stranded = escrow.balanceOfToken(address(s), address(lt));
    assertGt(stranded, 0);

    pons.setCreatorFeeRecipient(address(lt), creatorWallet); // documented owner power
    vm.warp(block.timestamp + 3 days);
```

```

pons.executeCreatorFeeRecipientChange(address[lt]);

vm.prank(creator);
vm.expectRevert(LaunchMismatch.selector);
s.bindLaunch(address[lt]);
vm.expectRevert(NotBound.selector);
s.claimLaunchTokensAndAccrue();
vm.prank(platform);
vm.expectRevert(NotBound.selector);
s.withdrawLaunchTokens();
assertEq(escrow.balanceOfToken(address[s], address[lt]), stranded, "no path ever claims it");
}

```

```
forge test --match-test "test_L03|test_V_L03" -w
```

Recommendation

Keep binding creator-only, since that is what stops third parties from binding the splitter to a junk launch. Drop the two state checks that can make it impossible.

The checks add no safety, because Pons itself rejects exit, buyback toggling and sweeps when the splitter is not the recipient. This is validated in

`test_FIX_L03_bindAfterOwnerRedirectRecoversStrandedTokens` and
`test_FIX_L03a_misconfiguredSplitterCanStillExit` :

```

if (
    launchToken == address[0] || launchToken == asset || !launch.exists || launch.token != launchToken
-   || launch.creatorFeeRecipient != address[this] || launch.pairToken != asset
) revert LaunchMismatch();

```

Also, have the launch flow create the splitter with the launch's actual pair token, so (a) cannot happen in the first place.

Team Response

Fixed.

I-01

● INFO

FIXED

Splitter Events and Views Do Not Expose the Full Revenue State

Description

The splitter's events and views do not describe its full revenue state.

The deterministic factory returns an existing splitter without an event. Vest releases and pool-fee sweeps emit no local event, while `_send()` emits `Paid` even when no value moves. Indexers that follow splitter events cannot distinguish deployment from reuse or observe each external revenue pull reliably.

Several unqualified views report only the quote-asset ledger, even though launch-token accounting exists separately. The ledgers are also accrual-lazy: a direct ERC-20 transfer changes the raw balance without changing owed or lifetime values until `_accrue()` runs, and it produces no splitter event.

Location of Affected Code

File: [contracts/src/CreatorFeeSplitter.sol#L217-L225](#)

```
function create(address quote, bytes32 salt) external returns (address splitter) {
    splitter = predict(msg.sender, quote, salt);
    if (splitter.code.length != 0) return splitter;
    splitter = address(new CreatorFeeSplitter(salt: _salt(msg.sender, quote, salt)){
        msg.sender, platform, quote, ponsFactory, feeEscrow
    });
    emit SplitterCreated(splitter, msg.sender, quote, salt);
}
```

The value-pull wrappers are at [contracts/src/FeeSplitterBase.sol#L229-L250](#). `_send()` emits `Paid` unconditionally at line 358.

File: [contracts/src/FeeSplitterBase.sol#L270-L292](#)

```
function platformOwed() external view returns (uint256) {
    return _ledgers[asset].platformOwed;
}

function lifetimeReceived() external view returns (uint256) {
    return _ledgers[asset].received;
}

function balance() external view returns (uint256) {
    return _balanceOf(asset);
}
```

File: [contracts/src/FeeSplitterBase.sol#L318-L328](#)

```
function _accrue(address trackedAsset) internal {
    Ledger storage l = _ledgers[trackedAsset];
    uint256 gross = _balanceOf(trackedAsset) - l.beneficiaryOwed - l.platformOwed;
    l.received += gross;
}
```

```
uint256 target = Math.mulDiv(l.received, PLATFORM_BPS, BPS_DENOMINATOR);
uint256 platformAmount = target - l.platformAllocated;
l.platformAllocated = target;
l.platformOwed += platformAmount;
l.beneficiaryOwed += gross - platformAmount;
emit Accrued(trackedAsset, gross, gross - platformAmount, platformAmount);
}
```

Impact

Event-indexed accounting can omit splitter reuse, external pulls, and direct receipts. Dashboards can also omit launch-token revenue or temporarily under-report beneficiary entitlements.

Withdrawals call `_accrue()` before payment, so the discrepancy does not lose funds.

Recommendation

Emit a distinct event when `create()` returns an existing splitter, including its current creator. Emit local events for vest releases, pool sweeps, and any internal settlement pulls added for [M-01]. Ensure helper refactoring emits each pull once. Skip `Paid` when its amount is zero or document the required filter.

Add quote-prefixed aliases for quote-only views and document that existing names exclude launch-token accounting. Expose `unaccounted(asset)` and tell indexers to reconcile raw balances alongside `Accrued` events.

Team Response

Fixed.

I-02

●

INFO

FIXED

Signer Rotation and Claim Resumption Coupled, Extending Outages and Ending Freezes Automatically

Description

The signer lifecycle couples two independent controls: key rotation and the incident claims freeze.

The owner can replace a pending signer but cannot cancel the change. Revoking the active signer also clears the pending signer, while the revocation flag can only be cleared by activating a replacement. If the owner schedules an unusable key and then revokes the signer, restoring the original key takes two delayed rotations because `scheduleSigner()` rejects the current signer.

The opposite transition has a separate problem. `activateSigner()` is intentionally permissionless after the delay, but it also clears `claimSignerRevoked`. Once the owner schedules a replacement during an incident, any account can end the claims freeze at the public activation timestamp. Clearing the flag also removes the condition that permits `cancelForIncident()`.

Earlier reviews describe activation clearing the revocation flag as intended recovery behavior [[docs/audits/2026-09-18-SHROOMS_V1_CONTRACT_AUDIT.md:104,364](#) ; [docs/audits/2026-09-18-SHROOMS_V1_SECOND_AUDIT.md:89](#)]. The 2026-09-21 review accepts that a revoked signer means at least two days without claims [[docs/CONTRACT_REVIEW_FIXES_2026-09-21.md:46](#)]. This issue does not dispute either decision. `cancelForIncident()` was added after those reviews, and because it requires `claimSignerRevoked`, permissionless activation now also closes the incident-refund window. The earlier reviews also do not cover the missing pending-only cancellation or the double rotation needed to restore a healthy key.

Location of Affected Code

File: [contracts/src/XTipEscrow.sol#L214-L240](#)

```
function scheduleSigner(address signer) external onlyOwner {
    if (signer == address(0) || signer == claimSigner) revert InvalidInput();
    if (signer == owner() || signer == pauser) revert RolesMustDiffer();
    pendingSigner = signer;
    signerEffectiveAt = block.timestamp + SIGNER_DELAY;
    emit ClaimSignerChangeScheduled(claimSigner, signer, signerEffectiveAt);
}

function revokeClaimSigner() external {
    if (msg.sender != owner() && msg.sender != pauser) revert Unauthorized();
    claimSignerRevoked = true;
    pendingSigner = address(0);
    signerEffectiveAt = 0;
    emit ClaimSignerRevoked(claimSigner, msg.sender);
}
```

File: [contracts/src/XTipEscrow.sol#L222-L230](#)

```
function activateSigner() external {
    if (pendingSigner == address(0) || block.timestamp < signerEffectiveAt) revert SignerChangeNotReady();
    emit ClaimSignerChanged(claimSigner, pendingSigner);
    claimSigner = pendingSigner;
    pendingSigner = address(0);
    signerEffectiveAt = 0;
    claimSignerRevoked = false;
}
```

`cancelForIncident()` requires `claimSignerRevoked == true` at [contracts/src/XTipEscrow.sol#L348-L355](#).

Impact

An operational mistake can block claims and new deposits for at least four days while the owner rotates away from an unusable key and then back to the original signer. During a real incident, permissionless activation lets any account end the refund campaign as soon as the replacement key matures.

Existing tips remain recoverable through expiry refunds. Custody is unaffected; availability and incident response are degraded.

Recommendation

Add an owner-only `cancelSignerChange()` that clears only `pendingSigner` and `signerEffectiveAt`, so a mistaken schedule can be dropped without revoking the active signer or rotating to another key.

Record the revoked signer in `revokeClaimSigner()` and remove the `claimSignerRevoked = false` write from `activateSigner()`. Add an owner-only `resumeClaims()` that clears `claimSignerRevoked` only when the active signer differs from the revoked one:

```
function revokeClaimSigner() external {
    if (msg.sender != owner() && msg.sender != pauser) revert Unauthorized();
    claimSignerRevoked = true;
    revokedSigner = claimSigner;
    pendingSigner = address(0);
    signerEffectiveAt = 0;
    emit ClaimSignerRevoked(claimSigner, msg.sender);
}

function resumeClaims() external onlyOwner {
    if (!claimSignerRevoked || claimSigner == revokedSigner) revert InvalidInput();
    claimSignerRevoked = false;
    emit ClaimsResumed(claimSigner);
}
```

A revoked key cannot be re-enabled without completing a delayed rotation, and the pauser keeps its stop-only role. Keep `activateSigner()` permissionless after the delay, so a replacement can mature while `cancelForIncident()` remains available and claims resume only when the owner ends the incident.

Team Response

Fixed.

Exit Lifecycle Checks Are Inconsistent Before and After the First Exit

Description

The splitter's exit checks are inconsistent at both ends of the lifecycle.

Before exit, `transferCreator()` rejects the platform address to preserve role separation, but `scheduleExit()` allows the creator to choose the platform as the final recipient. This lets the creator donate its complete future revenue share through a route that the ordinary role-transfer function prohibits.

After a successful exit, the splitter records no terminal state. The creator can schedule another exit locally. Execution later reaches the launcher and reverts because the splitter is no longer the current recipient. Transaction rollback restores the local schedule, allowing repeated attempts and misleading scheduling events.

Location of Affected Code

File: [contracts/src/CreatorFeeSplitter.sol#L101-L106](#)

```
function transferCreator(address next) external {
    if (msg.sender != creator) revert Unauthorized();
    if (next == platform || next == address(this) || next == creator) revert RolesMustDiffer();
    pendingCreator = next;
    emit CreatorTransferStarted(creator, next);
}
```

File: [contracts/src/CreatorFeeSplitter.sol#L135-L142](#)

```
function scheduleExit(address recipient) external {
    if (msg.sender != creator) revert Unauthorized();
    _boundToken();
    if (recipient == address(0) || recipient == address(this)) revert InvalidRecipient();
    exitRecipient = recipient;
    exitAt = block.timestamp + EXIT_DELAY;
    emit ExitScheduled(recipient, exitAt);
}
```

`contracts/src/CreatorFeeSplitter.sol#L135-L162` contains the complete exit state machine but has no terminal `exited` flag.

Impact

The platform-address case is creator-only self-harm: the creator can forfeit its 85% share of future covered revenue. Repeated post-exit schedules do not move funds, but they waste gas and create local events for an action the launcher can no longer complete.

Recommendation

Either reject `recipient == platform` in `scheduleExit()` or document it as an intentional donation path.

Optionally, add an `exited` flag and reject future schedules. Write the flag only after the external handoff succeeds, so a failed settlement check introduced for [M-01] or a failed launcher call cannot mark the splitter terminal. If the current external terminality check is intentional, document that later schedules are expected to fail at the launcher.

Team Response

Fixed.

I-04

● INFO

FIXED

Partial Claim Wrappers Accept Zero Amounts and Ignore the Escrow's Reported Payment

Description

The permissionless partial-claim wrappers send a caller-selected amount to the external fee escrow without rejecting zero. They also discard the amount returned by the escrow.

Balance-delta accounting recognizes the assets that actually arrive, so the current implementation does not over-allocate the splitter ledger. The wrapper's behavior and observability still depend on external escrow semantics for zero claims, over-claims, and partial payments.

Location of Affected Code

File: [contracts/src/FeeSplitterBase.sol#L202-L224](#)

```
function claimAmountAndAccrue(uint256 amount) external nonReentrant {
    if (asset == address(0)) feeEscrow.claim(amount);
    else feeEscrow.claimToken(asset, amount);
    _accrue(asset);
}

function claimLaunchTokenAmountAndAccrue(uint256 amount) external nonReentrant {
    address launchToken = _boundToken();
    feeEscrow.claimToken(launchToken, amount);
}
```

```
_accrue(launchToken);  
}
```

Impact

Against the current escrow, callers can waste gas on zero-value operations or encounter pass-through reverts without a local indication of the amount paid. A future escrow with different over-claim or partial-payment behavior could surprise callers, although the balance-delta accounting still prevents over-allocation.

Recommendation

Reject `amount == 0`, capture the escrow's returned payment, and include it in a local event. Add an integration test that pins production escrow behavior when the requested amount exceeds available credit.

Team Response

Fixed.

I-05

● INFO

FIXED

XTipEscrow **Cap Views Report Incompatible
Versions of Current Claim Capacity**

Description

The two public claim-cap views expose different incomplete versions of current bucket capacity.

`claimCapAvailable()` reports `claimCap - used`, but `_consumeClaimCap()` accepts any amount from an empty bucket. A legacy tip larger than a later reduced cap can therefore pass even when the view reports less headroom than the claim amount.

The public `claimBuckets()` getter has the opposite problem. It returns the last stored `used` value even though utilization drains virtually with time. Until another claim writes the updated value, direct consumers may see stale utilization indefinitely.

Neither surface is a reliable admission oracle on its own.

Location of Affected Code

File: [contracts/src/XTipEscrow.sol#L358-L363](#)

```
function claimCapAvailable(address asset) external view returns (uint256) {
    uint256 cap = assets[asset].claimCap;
    uint256 used = _bucketUsed(asset, cap);
    return used >= cap ? 0 : cap - used;
}
```

The empty-bucket exception is in `contracts/src/XTipEscrow.sol#L388-L393`.

File: [contracts/src/XTipEscrow.sol#L86](#)

```
mapping(address asset => ClaimBucket) public claimBuckets;
```

File: [contracts/src/XTipEscrow.sol#L376-L384](#)

```
function _bucketUsed(address asset, uint256 cap) private view returns (uint256 used) {
    ClaimBucket memory bucket = claimBuckets[asset];
    uint256 elapsed = block.timestamp - bucket.updatedAt;
    if (elapsed >= CLAIM_CAP_WINDOW) return 0;
    uint256 drained = Math.mulDiv(cap, elapsed, CLAIM_CAP_WINDOW);
    return bucket.used > drained ? bucket.used - drained : 0;
}
```

Impact

Monitoring can underestimate the next possible payment after a cap reduction or falsely report that the cap remains exhausted after it has drained. This can distort incident-response loss estimates and off-chain attester decisions. On-chain admission still uses the internally computed drained value.

Recommendation

Document `claimCapAvailable()` as ordinary drained headroom and `claimBuckets` as raw stored state. Add `bucketUsedNow(asset)` and an amount-aware `canClaimNow(asset, amount)` that use the same internal admission predicate as `_consumeClaimCap()`.

The amount-aware view must include the empty-bucket exception. If the admission rule for nonempty buckets is changed, the view must mirror that change as well. Ship the view and admission changes together.

Team Response

Fixed.

Reusing a Splitter Can Strand Revenue or Assign a Launch to the Wrong Creator

Description

Two factory behaviors make splitter reuse unsafe for integrations and direct launcher callers. The first-party launch and activation flows reject a pre-existing predicted splitter (`lib/pons/launch-preparation.ts:651-655`, `lib/pons/activation-preparation.ts:451-460`), so the application does not perform this reuse. Nothing in the contracts prevents it.

First, a splitter can bind only one launch token, while the launcher accepts any address as the creator-fee recipient for additional launches. Reusing a bound splitter lets quote revenue continue to arrive and split normally, which can hide the error. Token-side claims and vest releases still use the first bound token, leaving the second launch's token revenue inaccessible.

Second, deterministic `create()` returns existing code at the predicted address. The address depends on the original creator, quote asset, and salt, but the splitter's live creator role can be transferred. A wallet retry can therefore receive a valid splitter that the caller no longer controls. If that address is assigned to a new launch, revenue may go to the current role holder or become stranded through the one-token binding.

Location of Affected Code

File: [contracts/src/FeeSplitterBase.sol#L169-L179](#)

```
function bindLaunch(address launchToken) external {
  if (msg.sender != _binder()) revert Unauthorized();
  if (token != address(0)) revert AlreadyBound();
  IPonsFactory.Launch memory launch = ponsFactory.getLaunchedToken(launchToken);
  if (
    launchToken == address(0) || launchToken == asset || !launch.exists || launch.token != launchToken
    || launch.creatorFeeRecipient != address(this) || launch.pairToken != asset
  ) revert LaunchMismatch();
  token = launchToken;
  emit Bound(launchToken, msg.sender);
}
```

Every token claim and vest-release function uses `_boundToken()` and has no token argument.

File: [contracts/src/CreatorFeeSplitter.sol#L217-L225](#)

```
function create(address quote, bytes32 salt) external returns (address splitter) {
  splitter = predict(msg.sender, quote, salt);
}
```

```

if (splitter.code.length != 0) return splitter;
splitter = address[
  new CreatorFeeSplitter{salt: _salt(msg.sender, quote, salt)}(msg.sender, platform, quote, ponsFactory, feeEscrow)
];
emit SplitterCreated(splitter, msg.sender, quote, salt);
}

```

The mutable creator handoff is at `contracts/src/CreatorFeeSplitter.sol#L101-L116`.

Impact

An integration or direct launcher call that reuses a bound splitter leaves a second launch's memecoin-side fees and vesting value unclaimable through that splitter, while its quote revenue continues to split normally and masks the error. Reusing a role-rotated deterministic address assigns the launch's revenue to whoever currently holds the splitter's creator role. Correcting the recipient afterward depends on the current recipient's cooperation or on the Pons factory owner's delayed recipient override.

Recommendation

Document and enforce one splitter per launch in deployment tooling. Expose an `isServing(token)` helper. Before assigning a splitter to a launch, verify that it is unbound or bound to that exact token and that `splitter.creator() == expectedCreator`.

Document that `create()` may return an existing role-rotated splitter. Emit a `SplitterExists` event on the idempotent branch with the current creator so indexers can distinguish reuse from deployment.

Team Response

Acknowledged.

I-07

● INFO

FIXED

Splitter Deployment Does Not Verify That the Fee Escrow Belongs to the Factory

Description

The splitter constructors verify that the configured factory and fee escrow contain code but never prove that the escrow belongs to that factory. A staging and production endpoint mix-up can therefore deploy an immutable splitter against a mismatched pair. Quote claims and direct token claims may return no revenue or revert while the real revenue remains behind an endpoint the splitter cannot change.

`FeeSplitterBase` also states that the Pons owner can redirect a launch's creator recipient after a timelock. That is accurate for the live Pons V2 factory, which exposes `setCreatorFeeRecipient()` and `executeCreatorFeeRecipientChange()` with a three-day timelock (`CREATOR_FEE_RECIPIENT_TIMELOCK` reads 259200 on chain). The in-repository launcher disables those selectors, so tests built on it never exercise the override. The comment does not say that this override belongs to Pons governance and that Shrooms cannot trigger it.

Location of Affected Code

File: [contracts/src/FeeSplitterBase.sol#L143-L151](#)

```
constructor(address treasury, address quote, address factory, address escrow) {
    if (treasury == address(0)) revert ZeroAddress();
    if (factory.code.length == 0 || escrow.code.length == 0) revert NotContract();
    if (quote != address(0) && quote.code.length == 0) revert NotContract();
    platform = treasury;
    asset = quote;
    ponsFactory = IPonsFactory(factory);
    feeEscrow = IPonsEscrow(escrow);
}
```

File: [contracts/src/CreatorFeeSplitter.sol#L198-L204](#) repeats the same validation in the factory constructor.

File: [contracts/src/FeeSplitterBase.sol#L101-L104](#)

```
/// Trust assumptions this contract cannot remove:
/// - Pons' owner can redirect any launch's creator recipient after its timelock.
```

`contracts/launcher/src/ShroomsV1LaunchFactory.sol#L896-L899` disables the same selectors in the in-repository launcher. The live factory at `0x7eD598BcEf8bd9Edd8C97A195C6d13f40801EC7e` keeps them.

Impact

A deployment error can permanently strand quote or launch-token revenue and may stay hidden until launch activity begins. The splitter has no mutable endpoint configuration to repair it. The Pons owner override can move future fees to another recipient, but it cannot fix a splitter pointed at the wrong escrow.

Recommendation

The live Pons V2 factory exposes `feeEscrow()`, which returns `0xd3AFEB2a57f70eF218Aa82451c51B2fb0416Ac9e` on chain. Add the getter to the minimal interface and require `factory.feeEscrow() == escrow` in both constructors. Pin the same relationship in mocks and deployment checks. If a supported factory lacks the getter, use an explicit versioned factory/escrow allowlist. Do not catch a failed getter and silently accept an unverified pair.

Keep the NatSpec's note on the owner override, and add that it is Pons governance on the live factory, that Shrooms cannot trigger it, and that the in-repository launcher disables it.

Team Response

Fixed.

I-08

● INFO

FIXED

FeeSplitterBase

Clears ERC-20 Liabilities Without Verifying Exact Payout

Description

`FeeSplitterBase` treats an ERC-20 payout as fully settled once `safeTransfer(to, amount)` succeeds. If a configured quote asset or bound launch token is fee-on-transfer, deflationary on outgoing transfers, or otherwise debits or sends a different amount than requested, the splitter can clear the owed ledger and emit a full `Paid` amount while the recipient receives less.

`_payPlatform()` and `_payBeneficiary()` zero the owed balance before calling `_send()`. Reverts roll the write back, but a successful non-exact ERC-20 transfer does not revert. `_send()` does not snapshot and compare either the recipient balance increase or the splitter balance decrease, so the ledger can diverge from what was actually delivered.

The existing protections do not cover this case:

- `SafeERC20.safeTransfer()` only establishes that the token call did not report failure; it does not prove exact receipt.
- The accounting write is revert-atomic if the token reverts, but non-exact successful transfers do not revert.
- `_accrue()` observes balances before payout, but payout itself can change balances by a non-nominal amount.

- The stricter exact-delta payout pattern used by `XTipEscrow._pay()` [`contracts/src/XTipEscrow.sol#L396`] is not reused in `FeeSplitterBase._send()`.
- The contract comments identify negative rebase as unsupported but also mention fee-on-transfer remainder accounting, which can make the exact-transfer boundary ambiguous for splitter assets.

The issue requires the splitter to handle a non-exact ERC-20 as its quote asset or bound launch token. If production strictly admits only reviewed exact-transfer assets, this remains an unsupported-asset risk. If a fee-on-transfer, deflationary, sender-extra-debit, rebasing, blacklistable, or upgradeable token is admitted, recipients can be underpaid or later accounting can be disrupted.

Location of Affected Code

File: [contracts/src/FeeSplitterBase.sol#L331-L359](#)

```
function _payPlatform(address trackedAsset) internal {
    _accrue(trackedAsset);
    Ledger storage l = _ledgers[trackedAsset];
    uint256 amount = l.platformOwed;
    l.platformOwed = 0;
    _send(trackedAsset, platform, amount);
}

function _payBeneficiary(address trackedAsset, address to) internal {
    _accrue(trackedAsset);
    Ledger storage l = _ledgers[trackedAsset];
    uint256 amount = l.beneficiaryOwed;
    l.beneficiaryOwed = 0;
    _send(trackedAsset, to, amount);
}
```

`_send()` at `contracts/src/FeeSplitterBase.sol#L349` performs ERC-20 payouts with `safeTransfer()` only.

Impact

The immediate recipient can be underpaid relative to the splitter ledger and emitted event. If the token debits more than the nominal amount from the splitter, the remaining balance may no longer cover the other party's recorded entitlement, causing future accrual or withdrawal attempts to revert until the asset is voluntarily topped up. The maximum realistic impact is bounded by the non-exact token revenue handled by that splitter asset.

Proof of Concept

1. Deploy or bind a splitter whose quote asset or launch token is a mock ERC-20 with a successful but non-exact `transfer()` behavior, such as burning a fee, sending less than requested, or debiting the sender by more than the nominal amount.
2. Credit revenue for that ERC-20 to the splitter.
3. Call the relevant accrual path so the splitter records creator and platform owed balances for the ERC-20.
4. Have the creator or platform call the matching withdrawal function.
5. Observe that the owed ledger for the caller is cleared and `Paid(asset, to, amount)` is emitted.
6. Observe that the recipient received less than `amount`, or that an extra-debit token reduced the splitter balance in a way that can interfere with later accounting for the other beneficiary.

Recommendation

Either restrict splitter quote and launch-token assets to reviewed immutable exact-transfer ERC-20s, or add exact payout checks in `FeeSplitterBase_send()` for ERC-20 assets. The stricter implementation should snapshot the recipient and splitter balances before transfer, require the recipient balance to increase by exactly `amount`, and require the splitter balance to decrease by exactly `amount`, reverting otherwise.

Team Response

Fixed.

I-09

● INFO

FIXED

The Owner Can Bypass the Pauser Veto and Redirect Every Live Tip to Itself

Description

`claimSigner()` authorizes the payment of every tip, including existing tips. The only safeguard against a malicious signer rotation is the 2-day `SIGNER_DELAY`, during which, per the NatSpec, "the pauser can veto it in the meantime through `revokeClaimSigner()`". That safeguard does not hold:

- `setPauser()` is `onlyOwner` and takes effect immediately, and `revokeClaimSigner()` checks the *current* `pauser`.
- `activateSigner()` is permissionless, and it clears `claimSignerRevoked`.
- `setAsset()` can raise `claimCap` to `type(uint256).max` immediately.
- Senders cannot take a tip back before it expires.

The attack proceeds as follows:

1. In one multisig batch, the owner, or whoever controls the owner key, replaces the pauser and schedules a signer it controls.
2. The honest pauser's veto now reverts with `Unauthorized`. A pre-emptive revoke does not help either, because activation clears it (`test_V_M02_preemptiveRevokeDoesNotHelp`).
3. After two days, anyone activates the new signer and the owner lifts the cap.
4. Every tip is claimed to an owner-controlled address.

This contradicts the contract's stated trust model:

"Nobody, including the owner, can withdraw tips to themselves. The owner's strongest power over funds is returning a tip to its own sender during a declared incident."

Location of Affected Code

File: [contracts/src/XTipEscrow.sol](#)

- `setPauser()` (L205-L210)
- `scheduleSigner()` (L214-L220)
- `activateSigner()` (L223-L230)
- `setAsset()` (L162-L170)
- NatSpec at L20-L26 and L212-L213

```
function activateSigner() external {
    if (pendingSigner == address(0) || block.timestamp < signerEffectiveAt) revert SignerChangeNotReady();
    emit ClaimSignerChanged(claimSigner, pendingSigner);
    claimSigner = pendingSigner;
    pendingSigner = address(0);
    signerEffectiveAt = 0;
    claimSignerRevoked = false;
}
```

Impact

A malicious or compromised owner key can take:

- every tip with more than `SIGNER_DELAY` (2 days) left before expiry;
- every tip deposited during those 2 days, since deposits stay open.

Tips that expire within the delay escape: they fall back to the sender

(`test_V_M02_tipsExpiringWithinDelayEscape`). In the PoC, 3×100 ETH is drained. The pauser, presented as the independent check, cannot prevent it.

Proof of Concept

```
function test_MO2_ownerDisablesVetoAndDrainsAllTips() public {
    uint64 expiry = uint64(block.timestamp + 30 days);
    bytes32[3] memory ids;
    for (uint256 i; i < 3; ++i) {
        ids[i] = _tipNative(alice, 100 ether, expiry);
    }

    uint256 evilPk = 0xBAD;
    address loot = makeAddr("ownerControlledWallet");

    // One multisig batch: replace the pauser, then queue an owner-controlled attestor.
    vm.startPrank(admin);
    esc.setPauser(makeAddr("ownerPuppet"));
    esc.scheduleSigner(vm.addr(evilPk));
    vm.stopPrank();

    // The honest pauser sees the queued signer and tries the documented veto.
    vm.prank(pauser);
    vm.expectRevert(XTipEscrow.Unauthorized.selector);
    esc.revokeClaimSigner();

    vm.warp(block.timestamp + esc.SIGNER_DELAY());
    esc.activateSigner(); // permissionless
    vm.prank(admin);
    esc.setAsset(address(0), 0.01 ether, 100 ether, type(uint256).max, true); // cap lifted instantly

    for (uint256 i; i < 3; ++i) {
        _claim(evilPk, ids[i], loot);
    }
    assertEq(loot.balance, 300 ether, "owner-controlled wallet took every live tip");
    assertEq(address(esc).balance, 0);
}

function test_MO2_control_honestPauserVetoWorks() public {
    vm.prank(admin);
    esc.scheduleSigner(vm.addr(0xBAD));
    vm.prank(pauser);
    esc.revokeClaimSigner();
    vm.warp(block.timestamp + 2 days);
    vm.expectRevert(XTipEscrow.SignerChangeNotReady.selector);
    esc.activateSigner();
}

/// The honest pauser revoking pre-emptively (e.g. on seeing the batch in
/// the mempool) does not help: the batch still lands and activation clears
/// the revocation.
```

```

function test_V_M02_preemptiveRevokeDoesNotHelp() public {
    bytes32 id = _tipNative(alice, 100 ether, uint64(block.timestamp + 30 days));
    uint256 evilPk = 0xBAD;
    address loot = makeAddr("loot");

    vm.prank(pauser);
    esc.revokeClaimSigner();
    vm.startPrank(admin);
    esc.setPauser(makeAddr("puppet"));
    esc.scheduleSigner(vm.addr(evilPk));
    vm.stopPrank();

    vm.warp(block.timestamp + 2 days);
    esc.activateSigner();
    assertFalse(esc.claimSignerRevoked());
    _claim(evilPk, id, loot);
    assertEq(loot.balance, 100 ether);
}

```

```
forge test --match-test "test_M02|test_V_M02" -w
```

```
[PASS] test_M02_ownerDisablesVetoAndDrainsAllTips()
```

Recommendation

The owner controls who the pauser is, so any veto held by the pauser can only delay a malicious owner, not stop it. The fix that protects funds is to let the users at risk leave during the notice period. This is validated in `test_FIX_M02_senderCanExitDuringSignerChange` :

```

error NoSignerChangePending();

/// @notice While a new attester is queued, a sender may take back an active tip.
/// Turns SIGNER_DELAY into a real notice period for existing tips.
function cancelDuringSignerChange(bytes32 id) external nonReentrant {
    Tip storage tip = tips[id];
    if (msg.sender != tip.sender) revert Unauthorized();
    if (pendingSigner == address(0)) revert NoSignerChangePending();
    if (tip.status != Status.Active) revert TipNotActive();
    _refund(id, tip);
}

```

- **Defence in depth:** delay `setPauser()` and any increase of `claimCap` by at least `SIGNER_DELAY`, so a rotation cannot be combined with a silent pauser swap.
- **If the owner is meant to be fully trusted:** correct the NatSpec to say the owner can redirect live tips through a signer rotation.
- **Trade-off:** during a legitimate rotation, senders can also withdraw their tips, so recipients may lose tips they were expecting.

Team Response

Fixed.

I-10

INFO

FIXED

The Claim Cap Lets About Twice `claimCap` Out Within One Window

Description

`Asset.claimCap` is documented as the "Most that may be claimed within any `CLAIM_CAP_WINDOW`". It is also the stated loss limit for a stolen attestor key:

"the per-asset rolling claim cap bounds how much can leave before the pauser or owner revokes the key"

The implementation is a leaky bucket whose capacity and refill rate are **both** `claimCap` per window. A full burst of `claimCap` can be followed, inside the same window, by almost another `claimCap` of refill. Over any interval `T`, the maximum outflow is $\text{claimCap} \times (1 + T / \text{CLAIM_CAP_WINDOW})$. That is up to about twice the documented figure within one window.

Location of Affected Code

File: [contracts/src/XTipEscrow.sol](#)

- `_bucketUsed()` [L377-L384]
- `_consumeClaimCap()` [L388-L394]
- NatSpec at L21-L23, L61 and L160

```
function _bucketUsed(address asset, uint256 cap) private view returns (uint256 used) {
    ClaimBucket memory bucket = claimBuckets[asset];
    uint256 elapsed = block.timestamp - bucket.updatedAt;
    if (elapsed >= CLAIM_CAP_WINDOW) return 0;
    uint256 drained = Math.mulDiv(cap, elapsed, CLAIM_CAP_WINDOW);
    return bucket.used > drained ? bucket.used - drained : 0;
}
```

```
function _consumeClaimCap(address asset, uint256 amount) private {
    uint256 cap = assets[asset].claimCap;
    uint256 used = _bucketUsed(asset, cap);
    if (used != 0 && (amount > cap || used > cap - amount)) revert ClaimCapReached(used >= cap ? 0 : cap - used);
    claimBuckets[asset] = ClaimBucket(used + amount, uint64(block.timestamp));
}
```

Impact

- **Stolen key:** The loss a stolen attester key can cause is about twice what the documentation says.
- **In the PoCs:** 7,199 ETH leaves in 3,599 seconds against a documented 3,600 ETH/hour cap, and $4 \times$ `claimCap` leaves in 3 hours.
- **Operations:** operators who size caps from the NatSpec will underestimate how quickly they need to react.

Proof of Concept

```
function test_L04_nearlyTwiceTheCapInsideOneWindow() public {
    vm.prank(admin);
    esc.setAsset(address(0), 1 ether, 3600 ether, 3600 ether, true);
    uint64 expiry = uint64(block.timestamp + 30 days);
    bytes32 a = _tipNative(alice, 3600 ether, expiry);
    bytes32 b = _tipNative(alice, 3599 ether, expiry);
    address thief = makeAddr("stolenAttestorKeyHolder");

    uint256 t0 = block.timestamp;
    _claim(signerPk, a, thief);
    vm.warp(t0 + 3599);
    _claim(signerPk, b, thief);

    assertLt(block.timestamp - t0, esc.CLAIM_CAP_WINDOW());
    assertEq(thief.balance, 7199 ether); // 1.9997x the documented per-window maximum
}
```

*/// Generalised: over a response time $T \geq \text{window}$, total out is
 /// $\text{cap} * (1 + T/W)$, i.e. the burst is one extra cap, not a multiplier.*

```
function test_V_L04_boundOverLongerResponse() public {
    vm.prank(admin);
    esc.setAsset(address(0), 1 ether, 3600 ether, 3600 ether, true);
    uint64 expiry = uint64(block.timestamp + 30 days);
    address thief = makeAddr("thief");
    uint256 t0 = block.timestamp;

    // burst
    _claim(signerPk, _tipNative(alice, 3600 ether, expiry), thief);

    // then drain at the refill rate for 3 hours (1 ETH/s), in 600 s steps
    for (uint256 i; i < 18; ++i) {
```

```

vm.warp(block.timestamp + 600);
_claim(signerPk, _tipNative(alice, 600 ether, expiry), thief);
}
assertEq(block.timestamp - t0, 3 hours);
assertEq(thief.balance, 3600 ether + 3 * 3600 ether); // cap * [1 + T/W]
}

```

```
forge test --match-test "test_L04|test_V_L04" -vv
```

Recommendation

Correct the NatSpec to state the real limit:

```

/// @param claimCap Leaky-bucket capacity and hourly refill. Over any interval T, at most
/// claimCap * [1 + T / CLAIM_CAP_WINDOW] can be claimed; up to ~2x claimCap inside one window.

```

If a strict limit of L per window is needed, configure `claimCap = L / 2`. The existing code then enforces at most L per window. Because `setAsset()` requires `claimCap >= maximum`, this also caps the largest tip at $L / 2$.

Team Response

Fixed.

I-11

●

INFO

FIXED

Outgoing Creator Can Execute a Ready Exit Ahead of `acceptCreator()`

Description

`CreatorFeeSplitter.acceptCreator()` states that "Any exit the previous creator scheduled is cleared". The design notes make the same promise: accepting "clears any exit the previous creator scheduled" and "Pons keeps paying the splitter" ([docs/CONTRACT_REVIEW_FIXES_2026-09-21.md](#)).

That only holds if the acceptance lands first. A pending handoff does not freeze the exit schedule, and `acceptCreator()` takes no expected-state arguments. If an exit is ready when a new creator is nominated, for example, during a sale of the creator role, the outgoing creator can execute the exit before the nominee's `acceptCreator()` is mined. The acceptance still succeeds, and the nominee receives a splitter whose Pons creator fee recipient has already moved to the outgoing creator's address.

The same outcome is reachable without a race. The outgoing creator can cancel the nomination with `transferCreator(address[0])`, execute the exit, and nominate the same account again. The nominee's pending `acceptCreator()` then succeeds against the exited splitter.

Location of Affected Code

File: [contracts/src/CreatorFeeSplitter.sol#L108-L116](#)

```
/// @notice Accept the creator role. Unpaid creator revenue follows the role.  
/// Any exit the previous creator scheduled is cleared.  
function acceptCreator() external {  
    if (msg.sender != pendingCreator) revert Unauthorized();  
    emit CreatorTransferred(creator, msg.sender);  
    creator = msg.sender;  
    pendingCreator = address(0);  
    if (exitAt != 0) _clearExit();  
}
```

File: [contracts/src/CreatorFeeSplitter.sol#L154-L162](#)

```
function executeExitIfCurrent(address expectedRecipient, uint256 expectedExitAt) external nonReentrant {  
    if (msg.sender != creator) revert Unauthorized();  
    _requireCurrentExit(expectedRecipient, expectedExitAt);  
    if (block.timestamp < expectedExitAt) revert ExitNotReady();  
    exitAt = 0;  
    exitRecipient = address(0);  
    ponsFactory.transferCreatorFeeRecipient(token, expectedRecipient);  
    emit Exited(expectedRecipient);  
}
```

Impact

The incoming creator takes over a splitter that no longer receives Pons creator revenue and cannot restore the route. The treasury and splitter balances are unaffected beyond the effect of an ordinary exit. The affected party is an incoming creator who relied on the documented handoff behavior.

Proof of Concept

```
function test_exitBeforeAccept() public {  
    vm.prank(CREATOR);  
    s.scheduleExit(CW);  
    uint256 at = s.exitAt();  
    vm.warp(at);  
    vm.prank(CREATOR);  
    s.transferCreator(BUYER);  
  
    vm.prank(CREATOR); // lands before the nominee's acceptance  
    s.executeExitIfCurrent(CW, at);  
    vm.prank(BUYER);  
    s.acceptCreator();  
}
```

```

require(s.creator() == BUYER, "buyer is creator");
require(pons.getLaunchedToken(address[It]).creatorFeeRecipient == CW, "route gone");
}

function test_sameOutcomeWithoutRace() public {
    vm.prank(CREATOR);
    s.transferCreator(BUYER);
    vm.prank(CREATOR);
    s.scheduleExit(CW);
    uint256 at = s.exitAt();
    vm.warp(at);
    vm.prank(CREATOR);
    s.executeExitIfCurrent(CW, at);
    vm.prank(BUYER);
    s.acceptCreator();
    require(pons.getLaunchedToken(address[It]).creatorFeeRecipient == CW, "route gone");
}

```

Recommendation

Allow acceptance only while the splitter is still the Pons creator fee recipient:

```

error CreatorRecipientMoved();

function acceptCreator() external {
    if (msg.sender != pendingCreator) revert Unauthorized();
    address launchToken = token;
    if (
        launchToken != address(0)
        && ponsFactory.getLaunchedToken(launchToken).creatorFeeRecipient != address(this)
    ) revert CreatorRecipientMoved();
    emit CreatorTransferred(creator, msg.sender);
    creator = msg.sender;
    pendingCreator = address(0);
    if (exitAt != 0) _clearExit();
}

```

With this check, both sequences above revert at `acceptCreator()`, and the existing `CreatorFeeSplitter` unit and invariant tests continue to pass. The check also prevents a stale nominee from accepting after a completed exit.

The check must sit in `acceptCreator()`. Refusing to execute an exit while a handoff is pending is not sufficient, because the outgoing creator can cancel the nomination, exit, and nominate the same account again.

Team Response

Fixed.

I-12

● INFO

A Scheduled Exit Never Expires

Description

The only time check in `executeExitIfCurrent()` is `block.timestamp < expectedExitAt`. Once scheduled, an exit can be executed at any later time, even months later, with no fresh notice. The 7-day `EXIT_DELAY` is a one-time minimum; it says nothing about when the exit will actually happen.

Pons' own owner override expires after a 3-day execution window for exactly this reason.

Nothing marks the splitter as exited, either. If the creator-fee recipient is routed back to the splitter after an exit, the creator can schedule and use another exit.

Location of Affected Code

File: [contracts/src/CreatorFeeSplitter.sol#L154-L162](#)

```
function executeExitIfCurrent(address expectedRecipient, uint256 expectedExitAt) external nonReentrant {
    if (msg.sender != creator) revert Unauthorized();
    _requireCurrentExit(expectedRecipient, expectedExitAt);
    if (block.timestamp < expectedExitAt) revert ExitNotReady();
    exitAt = 0;
    exitRecipient = address(0);
    ponsFactory.transferCreatorFeeRecipient(token, expectedRecipient);
    emit Exited(expectedRecipient);
}
```

Impact

- The treasury cannot defend against the value redirection described in [M-01] by timing a final release or sweep.
- A standing exit option stays open indefinitely.

Proof of Concept

```
function test_I02_scheduledExitStaysExecutableIndefinitely() public {
    [CreatorFeeSplitter s, MockERC20 It,] = _boundGraduatedNativeSplitter();
    vm.prank(creator);
    s.scheduleExit(creatorWallet);
}
```

```

uint256 at = s.exitAt();
vm.warp(at + 365 days); // a year after the notice period ended, no new notice
vm.prank(creator);
s.executeExitIfCurrent(creatorWallet, at);
assertEq(pons.getLaunchedToken(address[1]).creatorFeeRecipient, creatorWallet);
}

```

```
forge test --match-test test_I02 -vv
```

Recommendation

Bound the execution window and make exit one-shot. This is validated in `test_FIX_I02_staleExitExpires` and `test_FIX_L01_pendingCurveFeesSettledBeforeHandover_andExitIsOneShot`:

```

error ExitExpired();
error AlreadyExited();

uint256 public constant EXIT_WINDOW = 3 days;
bool public exited;

// scheduleExit
if (exited) revert AlreadyExited();

// executeExitIfCurrent
if (block.timestamp > expectedExitAt + EXIT_WINDOW) revert ExitExpired();
exitAt = 0;
exitRecipient = address(0);
exited = true;

```

Team Response

N/A



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)